6. Procedure/Program:

Step 1: Update Ubuntu and Install Required Packages

1. Update the package repository:
   sudo apt update && sudo apt upgrade -y

2. Install Python & Required Libraries
   sudo apt install python3 python3-pip -y
   pip3 install cryptography pandas faker

   python3 --version
   pip3 list | grep -E "cryptography|pandas|faker"

Step 2: AES Encryption & Decryption

AES (Advanced Encryption Standard) is a symmetric encryption algorithm.

```
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~$ nano aes_key.py
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~$ ls
aes_key.py          ca_private.key  Desktop    Downloads  Pictures  preet   snap       user.crt  user_private.key  user_signed.crt
ca_certificate.crt  dac_deko        Documents  Music      pki_lab   Public  Templates  user.csr  user_public.key   Videos
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~$ mkdir exp6
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~$ cd exp6/
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$ nano aes_key.py
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$ ls
aes_key.py
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$ nano aes_encrypt.py
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$ ls
aes_encrypt.py  aes_key.py
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$ nano aes_decrypt.py
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$ ls
aes_decrypt.py  aes_encrypt.py  aes_key.py
```

1. Generate an AES Key : (Create nano aes_key.py)

aes_key.py 1

```python
from cryptography.fernet import Fernet

# Generate AES key
key = Fernet.generate_key()

# Save the key in a file
with open("aes_key.key", "wb") as key_file:
    key_file.write(key)

print(f"Generated AES Key: {key.decode()}")
```

2. Encrypt a Message (Create nano aes_encrypt.py)

aes_encrypt.py

```python
from cryptography.fernet import Fernet

# Load AES key
key = open("aes_key.key", "rb").read()
cipher = Fernet(key)

# Message to encrypt
message = "Confidential Data: Do not share!"

# Encrypt the message
encrypted_message = cipher.encrypt(message.encode())

# Save the encrypted message to a file
with open("encrypted_message.bin", "wb") as encrypted_file:
    encrypted_file.write(encrypted_message)

print(f"Encrypted Message: {encrypted_message.decode()}")
```

3. Decrypt the Message (Create nano aes_decrypt.py)

aes_decrypt.py

```python
from cryptography.fernet import Fernet

# Load AES key
key = open("aes_key.key", "rb").read()
cipher = Fernet(key)

# Read the encrypted message from the file
with open("encrypted_message.bin", "rb") as encrypted_file:
    encrypted_message = encrypted_file.read()

# Decrypt the message
decrypted_message = cipher.decrypt(encrypted_message).decode()
print(f"Decrypted Message: {decrypted_message}")
```

Step 3: RSA Encryption & Decryption

```
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$ nano rsa_key.py
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$ ls
aes_decrypt.py  aes_encrypt.py  aes_key.py  rsa_key.py
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$ nano rsa_encrypt.py
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$ ls
aes_decrypt.py  aes_encrypt.py  aes_key.py  rsa_encrypt.py  rsa_key.py
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$
```

1. Generate RSA Key Pair (Create nano rsa_key.py)

rsa_key.py 2

```python
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization

# Generate private key
private_key = rsa.generate_private_key(pub`ic_exponent=65537, key_size=2048)

# Save private key
with open("rsa_private.pem", "wb") as f:
    f.write(private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.TraditionalOpenSSL,
        encryption_algorithm=serialization.NoEncryption()
    ))

# Generate public key
public_key = private_key.public_key()
with open("rsa_public.pem", "wb") as f:
    f.write(public_key.public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo
    ))

print("RSA Key Pair Generated Successfully!")
```

2. Encrypt Data Using RSA Public Key (Create nano rsa_encrypt.py)

rsa_encrypt.py 2

```python
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import hashes, serialization

# Load the public key
public_key = serialization.load_pem_public_key(open("rsa_public.pem", "rb").read())

message = b"Secure Data Transfer"

# Encrypt using the public key
encrypted = public_key.encrypt(
    message,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Save the encrypted data to a file
with open("encrypted_data.bin", "wb") as encrypted_file:
    encrypted_file.write(encrypted)

print(f"Encrypted Data: {encrypted}")
```

3. Decrypt Data Using RSA Private Key (Create nano rsa_decrypt.py)

rsa_decrypt.py 2

```python
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import hashes, serialization

# Load the private key
private_key = serialization.load_pem_private_key(open("rsa_private.pem", "rb").read(), password=None)

# Read the encrypted data from the file
with open("encrypted_data.bin", "rb") as encrypted_file:
    encrypted = encrypted_file.read()

# Decrypt the message
decrypted = private_key.decrypt(
    encrypted,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

print(f"Decrypted Data: {decrypted.decode()}")
```

Step 4: Data Anonymization

1. Install Faker Library

   sudo apt install faker

2. Masking Sensitive Data (Anonymization) (Create nano mask.py)

mask.py 1

```python
import pandas as pd

data = {"SSN": ["123-45-6789", "987-65-4321", "555-44-3333"]}
df = pd.DataFrame(data)

# Masking SSN (showing only last 4 digits)
df["Masked_SSN"] = df["SSN"].str.replace(r"\d{3}-\d{2}", "***-**-", regex=True)
print(df)
```

2. Generate Fake Data for Privacy Protection

test.py 1

```python
from faker import Faker

fake = Faker()
for _ in range(5):
    print(fake.name(), "-", fake.email(), "-", fake.phone_number())
```

Step 5: Test and Verify Data Privacy

- Run encryption and anonymization scripts.
- Try to decrypt and view masked/anonymized data.
- Observe that data security policies prevent unauthorized access.

```
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$ python3 aes_key.py
Generated AES Key: Xl3GjpoQc_-c5HdbtlyvXy71IMt7ECjRkwi3b1D5saM=
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$ python3 aes_encrypt.py
Encrypted Message: gAAAAABn0RlY3bBREYungJGD2K7zBXaAcOSMWrBmYxHORO_Wg4YoU2Uq5SQQ1xTRRS1AMOFyzA3NlMBmkqxRbDg91-jNImLnjOgOqgaRWg9LyleQGqcyxRI70Uy4faQtMR8ESb-leoVP
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$ python3 aes_decrypt.py
Decrypted Message: Confidential Data: Do not share!
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$ python3 rsa_key.py
RSA Key Pair Generated Successfully!
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$ python3 rsa_encrypt.py
Encrypted Data: b'\xb6\xd77\xa4\xe2\xf0H\x14\xb0\x14]<\x0cW.\x9arh\n~\xd4\xf3|\x17yL\xcaU\xec]\xc6f<w,\xe3w"\xce\x00>%\xd9\xd4g\xb4\xc8\xa0\x02\x0c\x0c\x99\x1e\xfc&A\xcb\xc6b<a\x081\xa8\x86\x00e\x8f\x1a\x
92\xcd\xb2\x1b)\xad\x88\xf3\xc9os\xddj#\x8c\xee\xcc87\x1b\x99\x0c=k\xe6\xd2\xe4\x02\xf3\xfc\x90v\xae\xa3\xbcX\x8a\xd6:\xee\x95\xfe\xbd\xde\xe2\xcc\x94\xff\xe1\xec\xe3\rG\x04\xdf\x97\xf7\xe3\xb6\xb2\xad\x8
9\x08\xd8\x96\x80\\\xfb2\x91\xd5\xf8O{\xb4eK\xaf\x04\x1f\x7f\xfd\xcc5BM7\xb1t\xca\xe4\x1d\x97\xb3q\x05\x03\xf3\xc35-\x1d\xa6\xdd\x18\xc0\x07eqke\xad\xbd\xb1\x80\xc4\x7f\x018\xaa\xbd\x8d\x843\xa3\xc2\xc7\x
aa\x9f\\\xb4\xd8\xbd\xe7\xf4\xbf\xbb\xc2\x8f\x00\x16qd|n\xba\xca*\xf5\xa0\xed=\x10\xed\xc4\xff\xb1\xa2\x87\xb7\rW~\xa9A\x1d(>\xa2svIx\x83\xd2\x1e\xb7\xeb8F\xfe\xb1\x1f4\xd98\xc5'
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$ python3 rsa_decrypt.py
Decrypted Data: Secure Data Transfer
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$ python3 mask.py
           SSN    Masked_SSN
0  123-45-6789  ***-**--6789
1  987-65-4321  ***-**--4321
2  555-44-3333  ***-**--3333
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$ python3 test.py
Kenneth Lamb - fwallace@example.org - +1-324-267-7223x954
Chase Reid - turnercody@example.net - 001-812-383-6304x2092
Mark Martin - larry27@example.org - 001-621-759-5457x9286
Diane Wall - annecollins@example.com - (665)915-9538
Jeffrey Martin - mcclainshelby@example.com - 001-484-761-1787x5773
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/exp6$
```